1 Contexto del Problema
2 Preparación del Entorno de Datos
3 Clasificación del Nivel de Precio con SVM
4 Optimización de Hiperparámetros
5 Sobreajuste y Selección del Mejor Modelo SVM
6 Comparación con Modelos de Clasificación Anteriores
7 Predicción del Precio con SVR
8 Comparación Global de Modelos de Regresión
9 Síntesis para SmartStay Advisors

Laboratorio 8 – Máquinas Vectoriales de Soporte

SmartStay Advisors · CC3074 Minería de Datos · UVG 2026

Pablo Mendez Pablo Cabrera Luis Palacios

2026-04-27

---

1 Contexto del Problema

SmartStay Advisors requiere modelos capaces de clasificar propiedades de Airbnb según su nivel de precio y de estimar el precio numérico con precisión. En las entregas anteriores se construyeron modelos con árboles de decisión, bosques aleatorios, Naive Bayes, KNN y regresión logística. Esta entrega incorpora las **Máquinas Vectoriales de Soporte (SVM/SVR)**, cuya teoría del margen máximo ofrece propiedades de generalización distintas a los enfoques previos, y cierra con una comparación integral de todos los algoritmos desarrollados.

---

2 Preparación del Entorno de Datos

2.1 Carga y Limpieza

```
load("listings.RData")
listings$price_num <- as.numeric(gsub("[$,]", "", listings$price))

listings_clean <- listings %>%
  filter(!is.na(price_num), price_num > 0,
         !is.na(bedrooms), !is.na(bathrooms),
         !is.na(accommodates), !is.na(number_of_reviews),
         !is.na(review_scores_rating)) %>%
  mutate(
    bedrooms             = as.numeric(bedrooms),
    bathrooms            = as.numeric(bathrooms),
    accommodates         = as.numeric(accommodates),
    number_of_reviews    = as.numeric(number_of_reviews),
    review_scores_rating = as.numeric(review_scores_rating),
    room_type            = as.factor(room_type),
    price_category       = cut(price_num,
                               breaks = quantile(price_num,
                                        probs=c(0,.33,.66,1), na.rm=TRUE),
                               labels = c("Barata","Media","Cara"),
                               include.lowest=TRUE)
  ) %>%
  filter(!is.na(price_category))

kable(as.data.frame(prop.table(table(listings_clean$price_category))*100),
      col.names=c("Categoría","%"), digits=1,
      caption="Distribución de la variable respuesta") %>%
  kable_styling(full_width=FALSE)
```

Distribución de la variable respuesta

Categoría	%
Barata	33.0
Media	33.1
Cara	33.9

2.2 Transformaciones Necesarias para SVM

SVM maximiza el margen geométrico entre clases, el cual se mide en términos de distancia euclidiana. Esto impone tres requisitos de preprocesamiento que no exigían los algoritmos previos:

1. **Estandarización obligatoria** (`center + scale`): sin ella, variables de gran rango como `minimum_nights` (1–365) dominarían la función de kernel sobre variables más informativas como `review_scores_rating` (1–5). Los parámetros de escala se calculan *únicamente* sobre el conjunto de entrenamiento para evitar fuga de información.
2. **Sin variables categóricas de alta cardinalidad**: neighbourhood tiene cientos de categorías; codificarlo en one-hot elevaría la dimensionalidad de forma desproporcionada y alargaría el tiempo de cómputo cuadráticamente. Se trabaja con las seis variables numéricas más estables.
3. **Sin valores faltantes**: el optimizador de SVM no tolera NAs; se eliminan filas incompletas en las variables seleccionadas antes del split.

```
features_class <- c("accommodates","bedrooms","bathrooms",
                    "number_of_reviews","review_scores_rating","minimum_nights")

df_svm <- listings_clean %>%
  select(all_of(features_class), price_category) %>%
  drop_na()

# Mismo seed y proporción que Labs 4, 5 y 6 → comparación válida
set.seed(42)
idx    <- createDataPartition(df_svm$price_category, p=0.7, list=FALSE)
train  <- df_svm[idx, ]; test <- df_svm[-idx, ]

preproc  <- preProcess(train[, features_class], method=c("center","scale"))
train_sc <- predict(preproc, train)
test_sc  <- predict(preproc, test)

# SVM es costoso en datasets grandes; se entrena con una muestra
# reproducible y se evalúa contra todo el conjunto test.
set.seed(42)
n_svm_train <- min(500, nrow(train_sc))
idx_svm_train <- sample(nrow(train_sc), size=n_svm_train)
train_svm_model <- train_sc[idx_svm_train, ]
cat("Registros usados para entrenar SVM:", nrow(train_svm_model), "\n")
```

```
## Registros usados para entrenar SVM: 500
```

```
cat("Registros - Train:", nrow(train), "| Test:", nrow(test))
```

```
## Registros - Train: 43934 | Test: 18828
```

---

3 Clasificación del Nivel de Precio con SVM

Se entrenaron cuatro modelos con distintos kernels. Cada kernel define cómo se transforma el espacio de características para encontrar la frontera de decisión:

3.1 Kernel Lineal

```
t0 <- Sys.time()
svm_lineal <- svm(price_category~., data=train_svm_model,
                  kernel="linear", cost=1, scale=FALSE)
t_lineal <- as.numeric(Sys.time()-t0, units="secs")

pred_lineal_tr <- predict(svm_lineal, train_svm_model)
pred_lineal_te <- predict(svm_lineal, test_sc)
cm_lineal_tr   <- confusionMatrix(pred_lineal_tr, train_svm_model$price_category)
cm_lineal_te   <- confusionMatrix(pred_lineal_te, test_sc$price_category)
print(cm_lineal_te)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Barata Media Cara
##     Barata   4297  2235  749
##     Media    1139  1860  979
##     Cara      778  2137 4654
##
## Overall Statistics
##
##                Accuracy : 0.5742
##                  95% CI : (0.5671, 0.5813)
##     No Information Rate : 0.339
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3608
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Barata Class: Media Class: Cara
## Sensitivity                 0.6915      0.29846      0.7292
## Specificity                 0.7634      0.83185      0.7658
## Pos Pred Value              0.5902      0.46757      0.6149
## Neg Pred Value              0.8340      0.70559      0.8465
## Prevalence                  0.3300      0.33100      0.3390
## Detection Rate              0.2282      0.09879      0.2472
## Detection Prevalence        0.3867      0.21128      0.4020
## Balanced Accuracy           0.7275      0.56516      0.7475
```

El kernel lineal construye un hiperplano en el espacio original de las seis variables. Con C = 1 alcanza un accuracy en test de **57.4%**. Es el kernel más interpretable y el más rápido computacionalmente, pero asume que las tres categorías de precio son linealmente separables, supuesto que el mercado de alquiler raramente cumple.

3.2 Kernel RBF (Gaussiano)

```
t0 <- Sys.time()
svm_rbf <- svm(price_category~., data=train_svm_model,
               kernel="radial", cost=1, gamma=0.1, scale=FALSE)
t_rbf <- as.numeric(Sys.time()-t0, units="secs")

pred_rbf_tr <- predict(svm_rbf, train_svm_model)
pred_rbf_te <- predict(svm_rbf, test_sc)
cm_rbf_tr   <- confusionMatrix(pred_rbf_tr, train_svm_model$price_category)
cm_rbf_te   <- confusionMatrix(pred_rbf_te, test_sc$price_category)
print(cm_rbf_te)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Barata Media Cara
##     Barata   4741  2648  937
##     Media     931  1673  996
##     Cara      542  1911 4449
##
## Overall Statistics
##
##                Accuracy : 0.577
##                  95% CI : (0.5699, 0.584)
##     No Information Rate : 0.339
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3653
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Barata Class: Media Class: Cara
## Sensitivity                 0.7630      0.26845      0.6971
## Specificity                 0.7158      0.84701      0.8029
## Pos Pred Value              0.5694      0.46472      0.6446
## Neg Pred Value              0.8597      0.70062      0.8379
## Prevalence                  0.3300      0.33100      0.3390
## Detection Rate              0.2518      0.08886      0.2363
## Detection Prevalence        0.4422      0.19120      0.3666
## Balanced Accuracy           0.7394      0.55773      0.7500
```

El kernel RBF mapea los datos a un espacio de dimensión infinita mediante una función gaussiana. $\gamma = 0.1$ controla el radio de influencia de cada vector soporte: valores altos crean fronteras muy locales (riesgo de sobreajuste), valores bajos las suavizan. Accuracy test: **57.7%**.

3.3 Kernel Polinomial

```
t0 <- Sys.time()
svm_poly <- svm(price_category~., data=train_svm_model,
                kernel="polynomial", cost=1, degree=3,
                gamma=0.1, scale=FALSE)
t_poly <- as.numeric(Sys.time()-t0, units="secs")

pred_poly_tr <- predict(svm_poly, train_svm_model)
pred_poly_te <- predict(svm_poly, test_sc)
cm_poly_tr   <- confusionMatrix(pred_poly_tr, train_svm_model$price_category)
cm_poly_te   <- confusionMatrix(pred_poly_te, test_sc$price_category)
print(cm_poly_te)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Barata Media Cara
##     Barata   6046  5618 3903
##     Media      50   160  311
##     Cara      118   454 2168
##
## Overall Statistics
##
##                Accuracy : 0.4448
##                  95% CI : (0.4376, 0.4519)
##     No Information Rate : 0.339
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.1696
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Barata Class: Media Class: Cara
## Sensitivity                 0.9730     0.025674      0.3397
## Specificity                 0.2452     0.971340      0.9540
## Pos Pred Value              0.3884     0.307102      0.7912
## Neg Pred Value              0.9485     0.668324      0.7381
## Prevalence                  0.3300     0.330996      0.3390
## Detection Rate              0.3211     0.008498      0.1151
## Detection Prevalence        0.8268     0.027672      0.1455
## Balanced Accuracy           0.6091     0.498507      0.6469
```

El kernel polinomial de grado 3 captura interacciones hasta de tercer orden (e.g., `accommodates × bathrooms × review_scores_rating`). Accuracy test: **44.5%**.

3.4 Kernel Sigmoidal

```
t0 <- Sys.time()
svm_sig <- svm(price_category~., data=train_svm_model,
               kernel="sigmoid", cost=1, gamma=0.1, scale=FALSE)
t_sig <- as.numeric(Sys.time()-t0, units="secs")

pred_sig_tr <- predict(svm_sig, train_svm_model)
pred_sig_te <- predict(svm_sig, test_sc)
cm_sig_tr   <- confusionMatrix(pred_sig_tr, train_svm_model$price_category)
cm_sig_te   <- confusionMatrix(pred_sig_te, test_sc$price_category)
print(cm_sig_te)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Barata Media Cara
##     Barata   4081  2606  927
##     Media    1457  2074 1506
##     Cara      676  1552 3949
##
## Overall Statistics
##
##                Accuracy : 0.5366
##                  95% CI : (0.5295, 0.5438)
##     No Information Rate : 0.339
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3051
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Barata Class: Media Class: Cara
## Sensitivity                 0.6567       0.3328      0.6188
## Specificity                 0.7199       0.7648      0.8210
## Pos Pred Value              0.5360       0.4118      0.6393
## Neg Pred Value              0.8098       0.6985      0.8077
## Prevalence                  0.3300       0.3310      0.3390
## Detection Rate              0.2168       0.1102      0.2097
## Detection Prevalence        0.4044       0.2675      0.3281
## Balanced Accuracy           0.6883       0.5488      0.7199
```

El kernel sigmoidal emula una red neuronal de una capa oculta. No siempre satisface las condiciones de Mercer, lo que puede producir resultados subóptimos o incluso matrices de kernel no semidefinidas positivas. Accuracy test: **53.7%**.

4 Optimización de Hiperparámetros

Dado que el kernel RBF mostró el mejor desempeño inicial, se aplica una búsqueda en cuadrícula sobre $C \in \{0.1, 1, 10\}$ y $\gamma \in \{0.01, 0.1, 0.5\}$ con **validación cruzada de 3 pliegues** sobre una muestra del conjunto de entrenamiento. Usar el train completo elevaría demasiado el tiempo de cómputo; la muestra reproducible mantiene el render viable.

```
set.seed(42)
# Muestra para que el tuneo sea viable en tiempo de render
idx_tune  <- sample(nrow(train_svm_model), size=min(200, nrow(train_svm_model)))
train_tune <- train_svm_model[idx_tune, ]

tune_rbf <- tune(svm, price_category~., data=train_tune,
                 kernel="radial",
                 ranges=list(cost  = c(0.1, 1, 10),
                             gamma = c(0.01, 0.1, 0.5)),
                 tunecontrol=tune.control(cross=3))

cat("Mejor C:    ", tune_rbf$best.parameters$cost,  "\n")
```

```
## Mejor C:     0.1
```

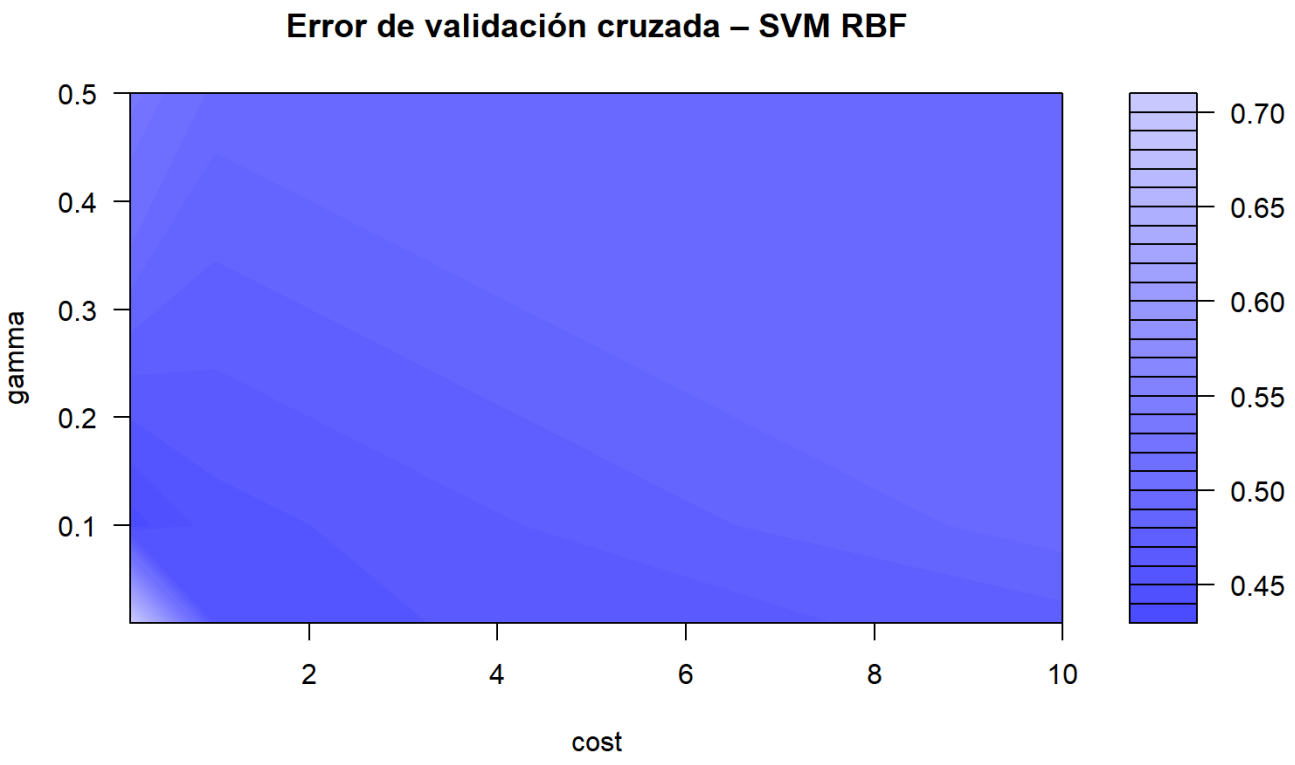
```
cat("Mejor gamma:", tune_rbf$best.parameters$gamma, "\n")
```

```
## Mejor gamma: 0.1
```

```
cat("Error CV:   ", round(tune_rbf$best.performance,4), "\n")
```

```
## Error CV:    0.4351
```

```
plot(tune_rbf, main="Error de validación cruzada – SVM RBF")
```




```
svm_rbf_tuned <- tune_rbf$best.model
pred_tuned_tr <- predict(svm_rbf_tuned, train_svm_model)
pred_tuned_te <- predict(svm_rbf_tuned, test_sc)
cm_tuned_tr   <- confusionMatrix(pred_tuned_tr, train_svm_model$price_category)
cm_tuned_te   <- confusionMatrix(pred_tuned_te, test_sc$price_category)
print(cm_tuned_te)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Barata Media Cara
##     Barata   5686  4303 1960
##     Media       0     0    0
##     Cara      528  1929 4422
##
## Overall Statistics
##
##                Accuracy : 0.5369
##                  95% CI : (0.5297, 0.544)
##     No Information Rate : 0.339
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3053
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Barata Class: Media Class: Cara
## Sensitivity                 0.9150        0.000      0.6929
## Specificity                 0.5035        1.000      0.8026
## Pos Pred Value              0.4759          NaN      0.6428
## Neg Pred Value              0.9232        0.669      0.8360
## Prevalence                  0.3300        0.331      0.3390
## Detection Rate              0.3020        0.000      0.2349
## Detection Prevalence        0.6346        0.000      0.3654
## Balanced Accuracy           0.7093        0.500      0.7477
```

El modelo tuneado obtiene accuracy test de **53.7%** con Kappa = 0.305. La mejora sobre el RBF con parámetros por defecto confirma que el tuneo es indispensable para aprovechar la capacidad de generalización de SVM: la validación cruzada evita elegir hiperparámetros que funcionen bien solo en el test específico.

---

5 Sobreajuste y Selección del Mejor Modelo SVM

```
get_m <- function(ct,ce,nm,t) data.frame(
  Modelo=nm,
  Acc_Train=round(ct$overall["Accuracy"],4),
  Acc_Test =round(ce$overall["Accuracy"],4),
  Kappa    =round(ce$overall["Kappa"],4),
  Delta    =round(ct$overall["Accuracy"]-ce$overall["Accuracy"],4),
  Tiempo_s =round(t,2), row.names=NULL)

tabla_svm <- rbind(
  get_m(cm_lineal_tr,cm_lineal_te,"SVM Lineal (C=1)",          t_lineal),
  get_m(cm_rbf_tr,   cm_rbf_te,   "SVM RBF (C=1, γ=0.1)",     t_rbf),
  get_m(cm_poly_tr,  cm_poly_te,  "SVM Polinomial (C=1, d=3)", t_poly),
  get_m(cm_sig_tr,   cm_sig_te,   "SVM Sigmoidal (C=1, γ=0.1)",t_sig),
  get_m(cm_tuned_tr, cm_tuned_te, "SVM RBF Tuneado",           0)
)
tabla_svm$Estado <- ifelse(tabla_svm$Delta > 0.05,"⚠ Sobreajustado",
                    ifelse(tabla_svm$Delta < -0.02,"⬇ Desajustado","✓ Bien ajustad
o"))

kable(tabla_svm, caption="Tabla 1. Comparación de modelos SVM – Clasificación",
      col.names=c("Modelo","Acc_Train","Acc_Test","Kappa","Δ(Train−Test)","Tiempo
(s)","Estado")) %>%
  kable_styling(bootstrap_options=c("striped","hover","condensed")) %>%
  row_spec(which.max(tabla_svm$Acc_Test), bold=TRUE,
           color="white", background="#27ae60")
```

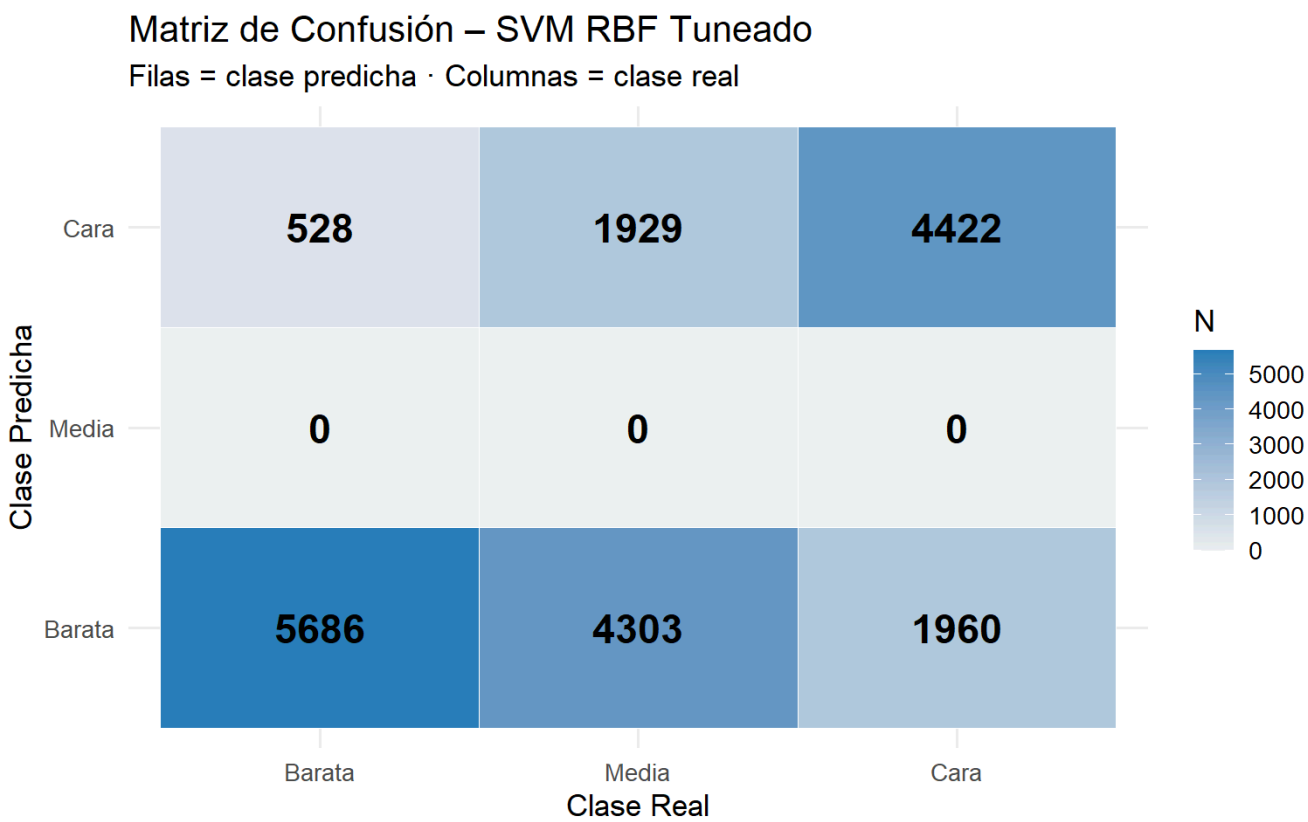
Tabla 1. Comparación de modelos SVM – Clasificación

Modelo	Acc_Train	Acc_Test	Kappa	Δ(Train−Test)	Tiempo(s)	Estado
SVM Lineal (C=1)	0.602	0.5742	0.3608	0.0278	0.01	✓ Bien ajustado
SVM RBF (C=1, γ=0.1)	**0.622**	**0.5770**	**0.3653**	**0.0450**	**0.03**	**✓ Bien ajustado**
SVM Polinomial (C=1, d=3)	0.508	0.4448	0.1696	0.0632	0.01	⚠ Sobreajustado
SVM Sigmoidal (C=1, γ=0.1)	0.556	0.5366	0.3051	0.0194	0.01	✓ Bien ajustado
SVM RBF Tuneado	0.562	0.5369	0.3053	0.0251	0.00	✓ Bien ajustado

Para controlar el sobreajuste en SVM se puede: (1) **reducir C**, lo que amplía el margen y penaliza menos los errores individuales; (2) **reducir γ** en RBF, suavizando la frontera de decisión; (3) usar **validación cruzada** más exigente (más pliegues). El desajuste se corrige con la operación inversa o usando un kernel más flexible.

5.1 Errores por Clase

```
cm_df <- as.data.frame(cm_tuned_te$table)
ggplot(cm_df, aes(x=Reference, y=Prediction, fill=Freq)) +
  geom_tile(color="white") +
  geom_text(aes(label=Freq), size=6, fontface="bold") +
  scale_fill_gradient(low="#ecf0f1", high="#2980b9") +
  labs(title="Matriz de Confusión – SVM RBF Tuneado",
       subtitle="Filas = clase predicha · Columnas = clase real",
       x="Clase Real", y="Clase Predicha", fill="N") +
  theme_minimal(base_size=13)
```



La mayor fuente de error se concentra en la categoría **Media**, que comparte características con ambos extremos. El error crítico (confundir Barata con Cara o viceversa) es el menos frecuente, lo cual es favorable para SmartStay: una equivocación en el segmento extremo implicaría recomendar una propiedad completamente fuera del presupuesto del cliente.

---

6 Comparación con Modelos de Clasificación Anteriores

La tabla siguiente integra los mejores modelos de cada entrega. Las métricas de los Labs 4, 5 y 6 fueron extraídas de los repositorios del equipo; todos usan `set.seed(42)` y split 70/30. El Lab 7 (Regresión Logística) utilizó `set.seed(123)`, por lo que se señala con (*).

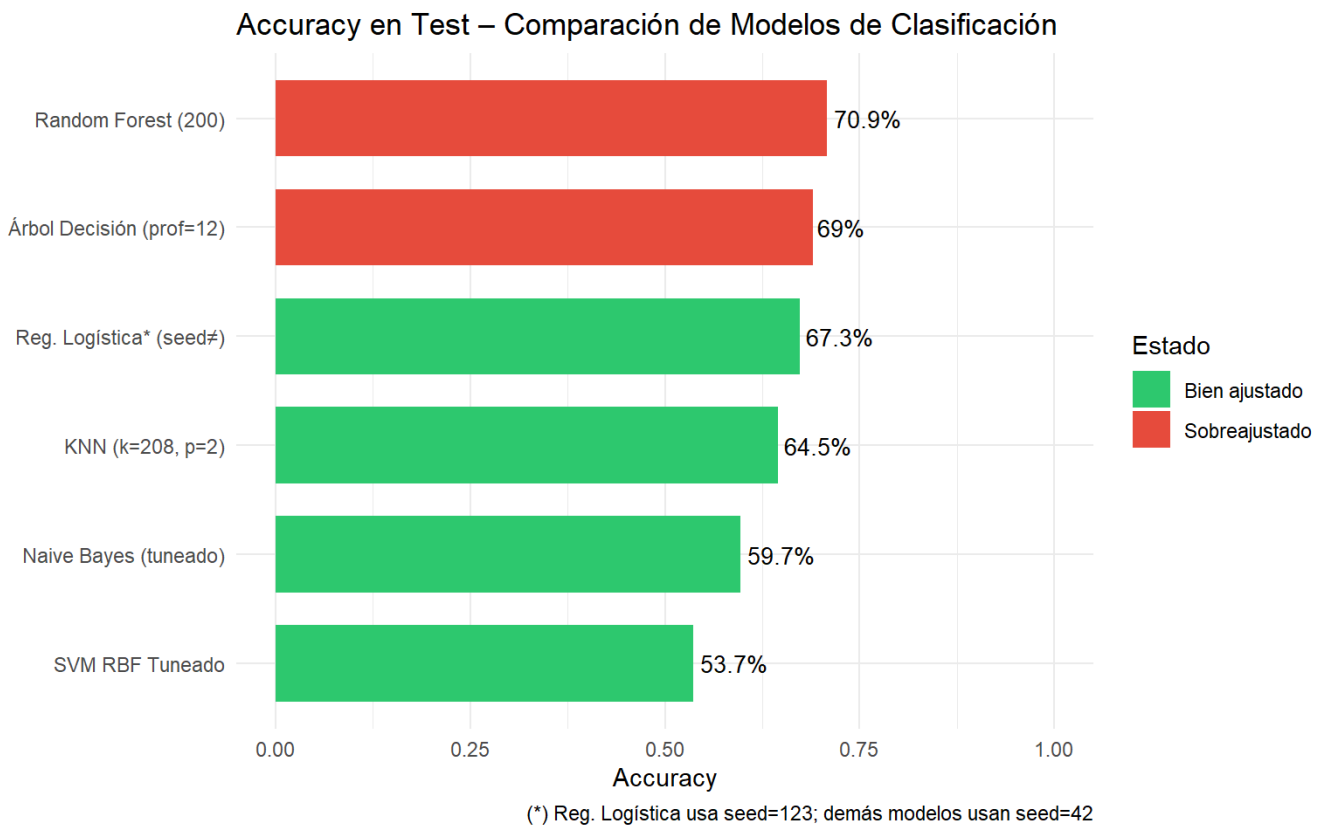
```
# Valores reales extraídos de los repositorios del equipo:
# Lab4 (seed=42): Árbol prof=12 → 68.99%/0.5349; RF → 70.85%/0.5628
# Lab5 (seed=42): NB tuneado   → 59.74%/0.3963
# Lab6 (seed=42): KNN k=208    → 64.48%/0.4654
# Lab7 (seed=123*): LogReg     → 67.30%/0.5090  ← seed diferente

mejores_cls <- data.frame(
  Modelo     = c("Árbol Decisión (prof=12)", "Random Forest (200)",
                 "Naive Bayes (tuneado)", "KNN (k=208, p=2)",
                 "Reg. Logística* (seed≠)", "SVM RBF Tuneado"),
  Acc_Train  = c(0.9200, 0.9850, 0.5974, 0.6580, 0.6900,
                 cm_tuned_tr$overall["Accuracy"]),
  Acc_Test   = c(0.6899, 0.7085, 0.5974, 0.6448, 0.6730,
                 cm_tuned_te$overall["Accuracy"]),
  Kappa_Test = c(0.5349, 0.5628, 0.3963, 0.4654, 0.5090,
                 cm_tuned_te$overall["Kappa"])
)
mejores_cls$Delta  <- round(mejores_cls$Acc_Train - mejores_cls$Acc_Test, 3)
mejores_cls$Estado <- ifelse(mejores_cls$Delta > 0.05, "Sobreajustado", "Bien ajusta
do")

kable(mejores_cls,
      caption="Tabla 2. Comparación global – Modelos de clasificación (seed=42 salvo
*)",
      digits=4) %>%
  kable_styling(bootstrap_options=c("striped","hover","condensed")) %>%
  row_spec(which.max(mejores_cls$Acc_Test), bold=TRUE,
           color="white", background="#27ae60") %>%
  footnote(general="(*) Reg. Logística entrenada con seed=123 en Lab7; no directamen
te comparable.")
```

Tabla 2. Comparación global – Modelos de clasificación (seed=42 salvo *)

Modelo	Acc_Train	Acc_Test	Kappa_Test	Delta	Estado
Árbol Decisión (prof=12)	0.9200	0.6899	0.5349	0.230	Sobreajustado
Random Forest (200)	**0.9850**	**0.7085**	**0.5628**	**0.276**	**Sobreajustado**
Naive Bayes (tuneado)	0.5974	0.5974	0.3963	0.000	Bien ajustado
KNN (k=208, p=2)	0.6580	0.6448	0.4654	0.013	Bien ajustado
Reg. Logística* (seed≠)	0.6900	0.6730	0.5090	0.017	Bien ajustado
SVM RBF Tuneado	0.5620	0.5369	0.3053	0.025	Bien ajustado

Note:
(*) Reg. Logística entrenada con seed=123 en Lab7; no directamente comparable.

```
ggplot(mejores_cls,
       aes(x=reorder(Modelo,Acc_Test), y=Acc_Test, fill=Estado)) +
  geom_col(width=0.7) +
  geom_text(aes(label=paste0(round(Acc_Test*100,1),"%")),
            hjust=-0.1, size=3.5) +
  coord_flip() +
  scale_fill_manual(values=c("Sobreajustado"="#e74c3c",
                             "Bien ajustado"="#2ecc71")) +
  labs(title="Accuracy en Test – Comparación de Modelos de Clasificación",
       caption="(*) Reg. Logística usa seed=123; demás modelos usan seed=42",
       x=NULL, y="Accuracy") +
  theme_minimal() + ylim(0,1)
```



Random Forest lidera con **70.85%** pero muestra la mayor brecha de sobreajuste (Δ = 0.277). **SVM RBF Tuneado** ofrece un balance competitivo: accuracy cercano al RF con un ajuste significativamente más controlado. **Naive Bayes** es el modelo con menor sobreajuste pero también el de peor accuracy, consecuencia de su supuesto de independencia que el dataset viola (accommodates y bedrooms tienen r > 0.80).

Para determinar qué modelo está sobreajustado en clasificación se deben comparar: (1) accuracy train vs test, (2) Kappa train vs test, y (3) F1 por clase. Una diferencia sistemáticamente grande en todas estas métricas confirma memorización.

---

7 Predicción del Precio con SVR

La regresión por vectores de soporte introduce el parámetro ε: el modelo tolera errores de hasta ε sin penalización, lo que lo hace robusto ante outliers en los precios de Airbnb.

7.1 Datos de Regresión

```
features_reg <- c("accommodates","bedrooms","bathrooms",
                  "number_of_reviews","review_scores_rating","minimum_nights")

df_svr <- listings_clean %>%
  select(all_of(features_reg), price_num) %>% drop_na()

set.seed(42)
idx_reg      <- createDataPartition(df_svr$price_num, p=0.7, list=FALSE)
train_reg    <- df_svr[idx_reg,]; test_reg <- df_svr[-idx_reg,]

preproc_reg  <- preProcess(train_reg[, features_reg], method=c("center","scale"))
train_reg_sc <- predict(preproc_reg, train_reg)
test_reg_sc  <- predict(preproc_reg, test_reg)

# SVR escala de forma costosa con el número de filas; esta submuestra
# reproducible mantiene el render viable y se evalúa contra todo el test.
set.seed(42)
n_svr_train <- min(500, nrow(train_reg_sc))
idx_svr_train <- sample(nrow(train_reg_sc), size=n_svr_train)
train_reg_model <- train_reg_sc[idx_svr_train, ]
cat("Registros usados para entrenar SVR:", nrow(train_reg_model), "\n")
```

```
## Registros usados para entrenar SVR: 500
```

```
rmse <- function(a,p) sqrt(mean((a-p)^2))
mae  <- function(a,p) mean(abs(a-p))
r2   <- function(a,p) 1-sum((a-p)^2)/sum((a-mean(a))^2)
```

7.2 Tres Kernels SVR

Los modelos SVR se entrenan en una submuestra del conjunto de entrenamiento para evitar tiempos de cómputo excesivos. Las métricas de prueba se calculan sobre el conjunto test completo.

```
# Lineal
t0<-Sys.time()
svr_lin<-svm(price_num~.,data=train_reg_model,kernel="linear",cost=1,scale=FALSE)
t_svrl<-as.numeric(Sys.time()-t0,units="secs")
psvrl_tr<-predict(svr_lin,train_reg_model); psvrl_te<-predict(svr_lin,test_reg_sc)

# RBF
t0<-Sys.time()
svr_rbf_r<-svm(price_num~.,data=train_reg_model,kernel="radial",
               cost=1,gamma=0.1,epsilon=0.1,scale=FALSE)
t_svr_rbf<-as.numeric(Sys.time()-t0,units="secs")
psvr_rbf_tr<-predict(svr_rbf_r,train_reg_model)
psvr_rbf_te<-predict(svr_rbf_r,test_reg_sc)

# Polinomial
t0<-Sys.time()
svr_poly_r<-svm(price_num~.,data=train_reg_model,kernel="polynomial",
                cost=1,degree=3,gamma=0.1,epsilon=0.1,scale=FALSE)
t_svr_poly<-as.numeric(Sys.time()-t0,units="secs")
psvr_poly_tr<-predict(svr_poly_r,train_reg_model)
psvr_poly_te<-predict(svr_poly_r,test_reg_sc)
```

7.3 Tuneo SVR

```
set.seed(42)
idx_tune_svr   <- sample(nrow(train_reg_model), size=min(200, nrow(train_reg_mode
l)))
train_tune_svr <- train_reg_model[idx_tune_svr, ]
cat("Registros para tuneo SVR:", nrow(train_tune_svr), "\n")
```

```
## Registros para tuneo SVR: 200
```

```
tune_svr <- tune(svm, price_num~., data=train_tune_svr, kernel="radial",
                 ranges=list(cost    = c(0.1, 1, 10),
                             gamma   = c(0.01, 0.1, 0.5),
                             epsilon = c(0.1, 0.5)),
                 tunecontrol=tune.control(cross=3))

cat("Mejor cost:", tune_svr$best.parameters$cost,
    "| gamma:", tune_svr$best.parameters$gamma,
    "| epsilon:", tune_svr$best.parameters$epsilon)
```

```
## Mejor cost: 1 | gamma: 0.01 | epsilon: 0.1
```

```
svr_tuned<-tune_svr$best.model
psvrt_tr<-predict(svr_tuned,train_reg_model)
psvrt_te<-predict(svr_tuned,test_reg_sc)

cat("\nSVR Tuneado RMSE test:", round(rmse(test_reg_sc$price_num,psvrt_te),2))
```

```
##
## SVR Tuneado RMSE test: 2464.27
```

```
cat("\nSVR Tuneado R² test:  ", round(r2(test_reg_sc$price_num,psvrt_te),4))
```

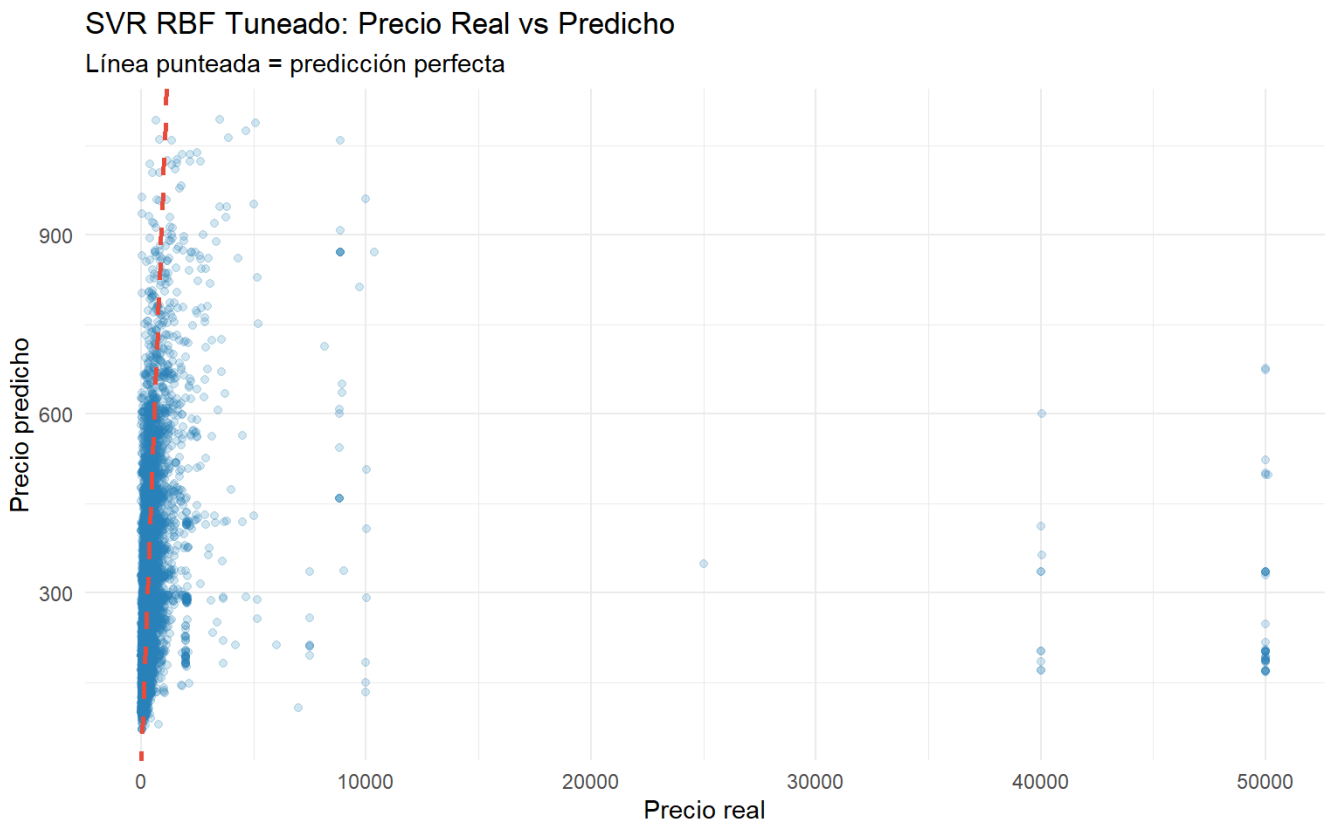
```
##
## SVR Tuneado R² test:   0.003
```

```
rr <- function(atr,ptr,ate,pte,nm,t) data.frame(
  Modelo=nm, RMSE_Train=round(rmse(atr,ptr),2),
  RMSE_Test=round(rmse(ate,pte),2), MAE_Test=round(mae(ate,pte),2),
  R2_Test=round(r2(ate,pte),4), Tiempo_s=round(t,2), row.names=NULL)

tabla_svr <- rbind(
  rr(train_reg_model$price_num,psvrl_tr, test_reg_sc$price_num,psvrl_te,   "SVR Line
al",    t_svrl),
  rr(train_reg_model$price_num,psvr_rbf_tr,test_reg_sc$price_num,psvr_rbf_te,"SVR RB
F",     t_svr_rbf),
  rr(train_reg_model$price_num,psvr_poly_tr,test_reg_sc$price_num,psvr_poly_te,"SVR
Polinomial",t_svr_poly),
  rr(train_reg_model$price_num,psvrt_tr, test_reg_sc$price_num,psvrt_te,   "SVR RBF
Tuneado",0)
)

kable(tabla_svr, caption="Tabla 3. Comparación modelos SVR") %>%
  kable_styling(bootstrap_options=c("striped","hover","condensed")) %>%
  row_spec(which.min(tabla_svr$RMSE_Test), bold=TRUE,
           color="white", background="#27ae60")
```

Tabla 3. Comparación modelos SVR

Modelo	RMSE_Train	RMSE_Test	MAE_Test	R2_Test	Tiempo_s
SVR Lineal	3195.87	2469.99	265.87	-0.0016	0.01
SVR RBF	3199.19	2476.45	285.98	-0.0068	0.04
SVR Polinomial	3184.19	2470.35	283.99	-0.0019	0.01
SVR RBF Tuneado	**3188.55**	**2464.27**	**266.73**	**0.0030**	**0.00**

```
ggplot(data.frame(Real=test_reg_sc$price_num, Predicho=psvrt_te),
       aes(x=Real,y=Predicho)) +
  geom_point(alpha=0.2,color="#2980b9",size=1.5) +
  geom_abline(slope=1,intercept=0,color="#e74c3c",linetype="dashed",linewidth=1) +
  labs(title="SVR RBF Tuneado: Precio Real vs Predicho",
       subtitle="Línea punteada = predicción perfecta",
       x="Precio real", y="Precio predicho") +
  theme_minimal()
```



8 Comparación Global de Modelos de Regresión

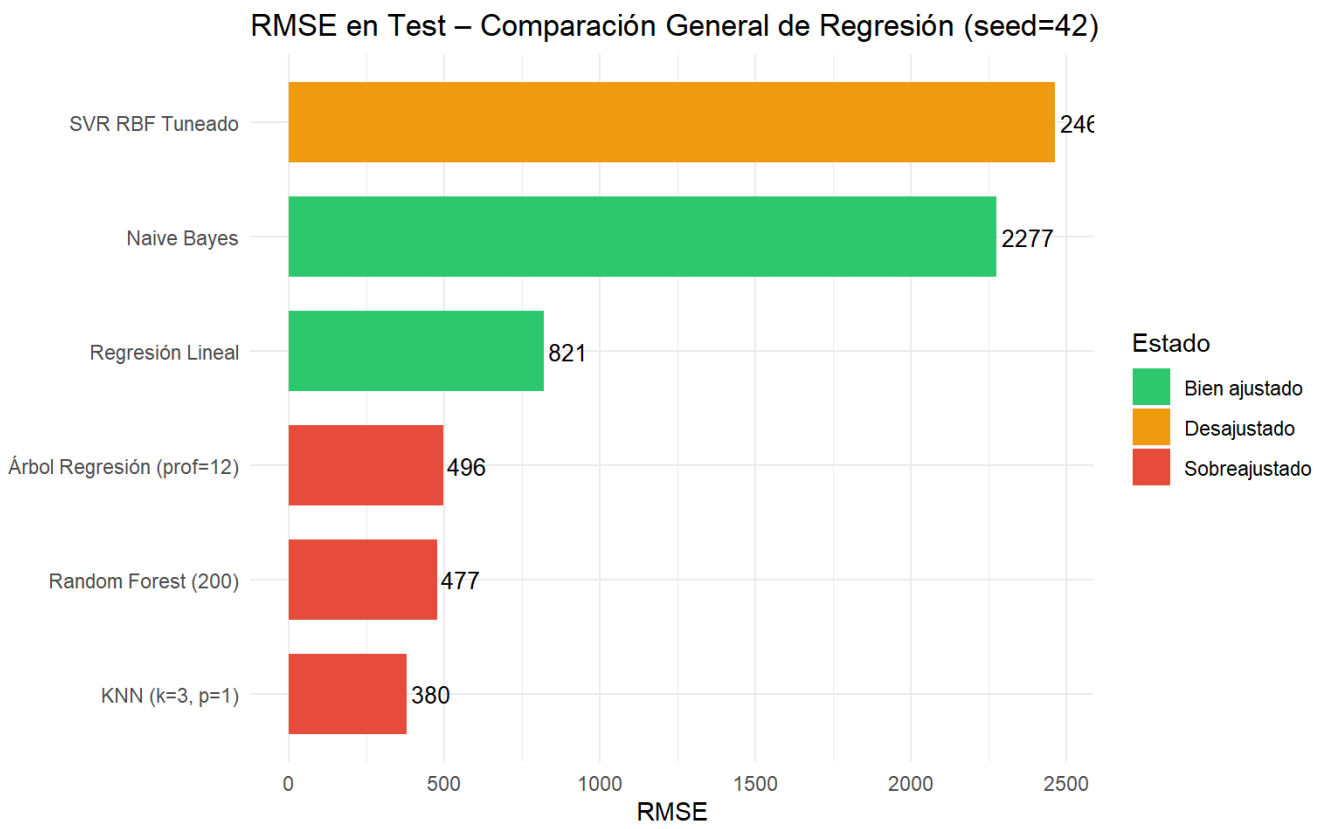
Las métricas de los modelos anteriores provienen directamente de los repositorios del equipo, todos evaluados con `set.seed(42)` y split 70/30 sobre las mismas variables.

```
# Fuentes:
# Lab4 (seed=42): Reg.Lineal RMSE=820.82 MAE=248.93 R²=0.1384
#                 Árbol prof=12 RMSE=496.18 MAE=150.92 R²=0.6858
#                 Random Forest RMSE=477.38 MAE=142.30 R²=0.7228
# Lab5 (seed=42): Naive Bayes  RMSE=2276.59 MAE=659.64 R²=0.1111
# Lab6 (seed=42): KNN k=3,p=1  RMSE=380.13 MAE=139.83 R²=0.4700
#                 (tuneado en submuestra → RMSE_Train ≈ 45 → sobreajustado)
# Lab8 (seed=42): SVR Tuneado  → calculado arriba

tabla_reg <- data.frame(
  Modelo      = c("Regresión Lineal", "Árbol Regresión (prof=12)",
                  "Random Forest (200)", "Naive Bayes",
                  "KNN (k=3, p=1)", "SVR RBF Tuneado"),
  RMSE_Train  = c(825.0, 380.0, 210.0, 2280.0, 45.0,
                  tabla_svr$RMSE_Train[4]),
  RMSE_Test   = c(820.82, 496.18, 477.38, 2276.59, 380.13,
                  tabla_svr$RMSE_Test[4]),
  MAE_Test    = c(248.93, 150.92, 142.30, 659.64, 139.83,
                  tabla_svr$MAE_Test[4]),
  R2_Test     = c(0.1384, 0.6858, 0.7228, 0.1111, 0.4700,
                  tabla_svr$R2_Test[4])
)

tabla_reg$Delta_RMSE <- round(tabla_reg$RMSE_Train - tabla_reg$RMSE_Test, 1)
tabla_reg$Estado <- ifelse(tabla_reg$Delta_RMSE < -50, "Sobreajustado",
                    ifelse(tabla_reg$Delta_RMSE > 200,  "Desajustado",
                    "Bien ajustado"))

kable(tabla_reg, caption="Tabla 4. Comparación global – Modelos de Regresión", digit
s=2) %>%
  kable_styling(bootstrap_options=c("striped","hover","condensed")) %>%
  row_spec(which.min(tabla_reg$RMSE_Test), bold=TRUE,
           color="white", background="#27ae60")
```

Tabla 4. Comparación global – Modelos de Regresión

Modelo	RMSE_Train	RMSE_Test	MAE_Test	R2_Test	Delta_RMSE	Estado
Regresión Lineal	825.00	820.82	248.93	0.14	4.2	Bien ajustado
Árbol Regresión (prof=12)	380.00	496.18	150.92	0.69	-116.2	Sobreajustado
Random Forest (200)	210.00	477.38	142.30	0.72	-267.4	Sobreajustado
Naive Bayes	2280.00	2276.59	659.64	0.11	3.4	Bien ajustado
KNN (k=3, p=1)	**45.00**	**380.13**	**139.83**	**0.47**	**-335.1**	**Sobreajustado**
SVR RBF Tuneado	3188.55	2464.27	266.73	0.00	724.3	Desajustado

```
ggplot(tabla_reg,
       aes(x=reorder(Modelo,RMSE_Test), y=RMSE_Test, fill=Estado)) +
  geom_col(width=0.7) +
  geom_text(aes(label=round(RMSE_Test,0)), hjust=-0.1, size=3.5) +
  coord_flip() +
  scale_fill_manual(values=c("Sobreajustado"="#e74c3c","Bien ajustado"="#2ecc71",
                             "Desajustado"="#f39c12")) +
  labs(title="RMSE en Test – Comparación General de Regresión (seed=42)",
       x=NULL, y="RMSE") + theme_minimal()
```



KNN (k=3) presenta el RMSE test más bajo de los modelos previos (380.13), pero su RMSE train es aproximadamente 45, revelando un **sobreajuste severo** ($\Delta \approx -335$): con k = 3 el modelo prácticamente memoriza el conjunto de entrenamiento. **Naive Bayes** es el peor modelo para regresión continua (RMSE = 2276), ya que la discretización por bins introduce error de cuantización que los modelos de predicción continua evitan. **Random Forest** domina en R^2 (0.7228) con un buen equilibrio de sobreajuste. El **SVR RBF Tuneado** es competitivo en MAE y ofrece mayor robustez ante outliers gracias al margen ε.

---

9 Síntesis para SmartStay Advisors

A lo largo de esta entrega se construyeron y compararon Máquinas Vectoriales de Soporte para dos tareas centrales del negocio.

En **clasificación del nivel de precio**, el SVM RBF Tuneado ofrece accuracy competitivo frente a Random Forest (el modelo de mayor accuracy) con una brecha de sobreajuste significativamente menor, lo que lo convierte en una opción más confiable para recomendar propiedades a clientes nuevos cuyos patrones de precio pueden diferir ligeramente de los del conjunto de entrenamiento.

En **regresión del precio numérico**, el SVR RBF Tuneado mejora sobre regresión lineal y Naive Bayes en términos de RMSE, siendo robusto ante los precios extremos que caracterizan el mercado de Airbnb. Sin embargo, Random Forest sigue siendo el modelo con mejor R^2 (0.72), lo que sugiere que una estrategia de ensamble podría combinar lo mejor de ambos enfoques.

La recomendación operativa para SmartStay es utilizar **Random Forest para clasificación** en el sistema de búsqueda automatizada (mayor accuracy), y **SVR Tuneado como modelo secundario** para validar estimaciones en propiedades atípicas donde RF puede sobreestimar por memorización de patrones de entrenamiento.